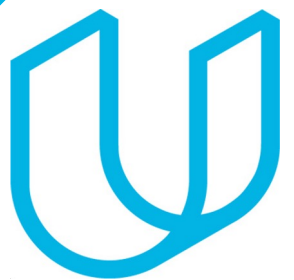


HR Database Design

Ryan Miles – 4/25/25



Business Scenario

Business requirement

Tech ABC Corp saw explosive growth with a sudden appearance onto the gaming scene with their new AI-powered video game console. As a result, they have gone from a small 10 person operation to 200 employees and 5 locations in under a year. HR is having trouble keeping up with the growth, since they are still maintaining employee information in a spreadsheet. While that worked for ten employees, it has become increasingly cumbersome to manage as the company expands.

As such, the HR department has tasked you, as the new data architect, to design and build a database capable of managing their employee information.

Dataset

The [HR dataset](#) you will be working with is an Excel workbook which consists of 206 records, with eleven columns. The data is in human readable format, and has not been normalized at all. The data lists the names of employees at Tech ABC Corp as well as information such as job title, department, manager's name, hire date, start date, end date, work location, and salary.

IT Department Best Practices

The IT Department has certain Best Practices policies for databases you should follow, as detailed in the [Best Practices document](#).



Step 1

Data Architecture
Foundations

Step 1: Data Architecture Foundations

Hi,

Welcome to Tech ABC Corp. We are excited to have some new talent onboard. As you may already know, Tech ABC Corp has recently experienced a lot of growth. Our AI powered video game console WOPR has been hugely successful and as a result, our company has grown from 10 employees to 200 in only 6 months (and we are projecting a 20% growth a year for the next 5 years). We have also grown from our Dallas, Texas office, to 4 other locations nationwide: New York City, NY, San Francisco, CA, Minneapolis, MN, and Nashville, TN.

While this growth is great, it is really starting to put a strain on our record keeping in HR. We currently maintain all employee information on a shared spreadsheet. When HR consisted of only myself, managing everyone on an Excel spreadsheet was simple, but now that it is a shared document I am having serious reservations about data integrity and data security. If the wrong person got their hands on the HR file, they would see the salaries of every employee in the company, all the way up to the president.

After speaking with Jacob Lauber, the manager of IT, he suggested I put in a request to have my HR Excel file converted into a database. He suggested I reach out to you as I am told you have experience in designing and building databases. When you are building this, please keep in mind that I want any employee with a domain login to be have read only access the database. I just don't want them having access to salary information. That needs to be restricted to HR and management level employees only. Management and HR employees should also be the only ones with write access. By our current estimates, 90% of users will be read only.

I also want to make sure you know that am looking to turn my spreadsheet into a live database, one I can input and edit information into. I am not really concerned with reporting capabilities at the moment. Since we are working with employee data we are required by federal regulations to maintain this data for at least 7 years; additionally, since this is considered business critical data, we need to make sure it gets backed up properly.

As a final consideration. We would like to be able to connect with the payroll department's system in the future. They maintain employee attendance and paid time off information. It would be nice if the two systems could interface in the future

I am looking forward to working with you and seeing what kind of database you design for us.

Thanks,
Sarah Collins
Head of HR

Data Architect Business Requirement

- **Purpose of the new database:**

The purpose of this new database is to keep up with a growing business, new employees coming in, many locations opening up and keeping business critical data secure and maintaining its integrity. Additionally, this data needs to comply with the law.

- **Describe current data management solution:**

Currently the Head of HR manages their data in a single spreadsheet that is shared with other HR and management employees.

- **Describe current data available:**

Currently the data available is employee data consisting of: employee ID's, Employee names, employee emails, hire dates, job titles, salaries, departments, managers info, start date and end date, office location, cities, states, and employee education levels

- **Additional data requests:**

Backup schedule, connecting to payroll department systems, provide new users read access only

- **Who will own/manage data**

HR will own and manage the data

- **Who will have access to database**

HR and Mgmt employees will have write access, the rest (90%) of the employees will have read access only to certain levels of data

Data Architect Business Requirement

- **Estimated size of database**

200 rows x 14 columns

30KB

- **Estimated annual growth**

20%+ per year for the next 5 years

- **Is any of the data sensitive/restricted**

Salary info is sensitive and should be restricted to being seen by about 90% of employees.

Data Architect Technical Requirement

- **Justification for the new database**

TO maintain business critical data security and integrity

To keep up HR operations and systems with a quickly growing business, moving away from manual operations

- **Database objects**

List the database objects (tables, views, special procedures) that will be created for the database.

-Tables,

Hint - you may want to circle back to this answer after completing the logical ERD in step 2.

- **Data ingestion**

ETL and/or API

Data Architect Technical Requirement

- **Data governance (Ownership and User access)**

Ownership: HR Department

User Access: non HR employees will have read access to their data, HR & Mgmt employees to have r/w access and access to all salary info

- **Scalability**

Replication should be used

- **Flexibility**

The needs of this database should remain pretty static. Users should remain all employees and HR/Mgmt employees, the platform should not need much changing in the next 5 years

- **Storage & retention**

Storage (disk or in-memory): Disk

Retention: 7 years

- **Backup**

Critical: Backup schedule is full backup 1x per week, incremental backup daily.



Step 2

Relational Database Design

Step 2: Relational Database Design

This step is where you will go through the process of designing a new database for Tech ABC Corp's HR department. Using the [dataset](#) provided, along with the requirements gathered in step one, you are going to develop a relational database set to the 3NF.

Using Lucidchart, you will create 3 entity relationship diagrams (ERDs) to show how you developed the final design for your data.

You will submit a screenshot for each of the 3 ERDs you create. You will find detailed instructions for developing each of the ERDs over the next several pages.

ERD

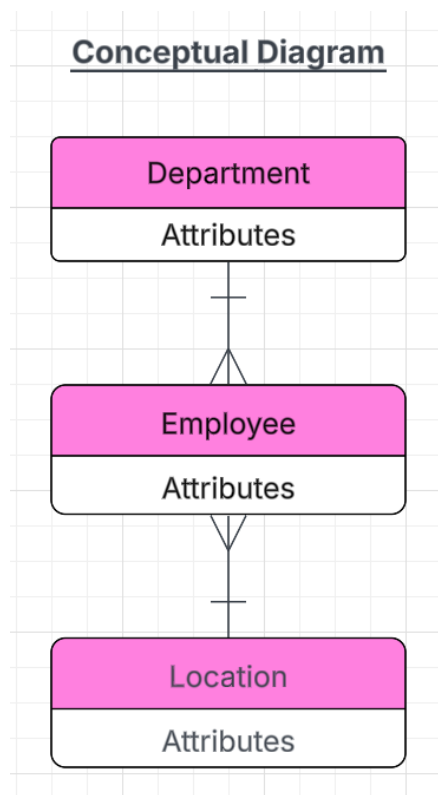
- **Conceptual**

This is the most general level of data modeling. At the conceptual level, you should be thinking about creating entities that represent business objects for the database. Think broadly here.

Attributes (or column names) are not required at this point, but relationship lines are required (although Crow's foot notation is not needed at this level). Create at least three entities for this model; thinking about the 3NF will aid you in deciding the type of entities to create.

Use Lucidchart's built-in template for DBMS ER Diagram UML.

**** Replace example screenshot below with your response**



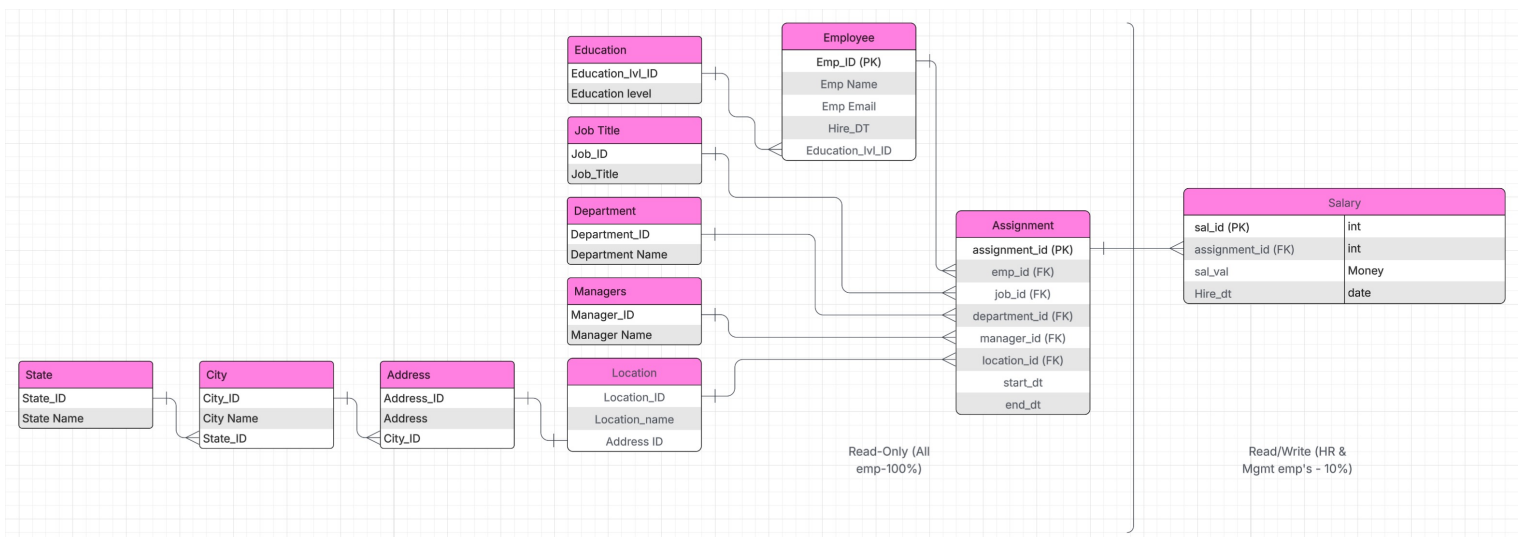
ERD

- **Logical**

The logical model is the next level of refinement from the conceptual ERD. At this point, you should have normalized the data to the 3NF. Attributes should also be listed now in the ERD. You can still use human-friendly entity and attribute names in the logical model, and while relationship lines are required, Crow's foot notation is still not needed at this point.

Use Lucidchart's built-in template for DBMS ER Diagram UML.

**** Replace example screenshot below with your response**



ERD

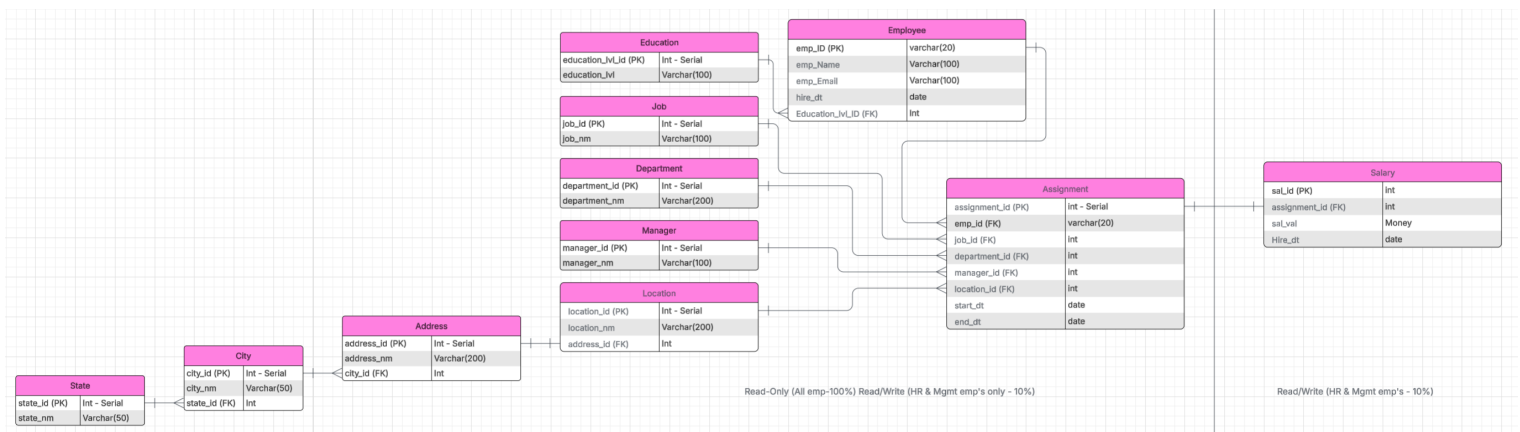
- Physical

The physical model is what will be built in the database. Each entity should represent a database table, complete with column names and data types.

Primary keys and foreign keys should also be represented here. Primary keys should be in bold type with the (PK) designation following the field name.

Foreign keys should be in normal type face, but have the designation (FK) after the column name. Finally, in the physical model, Crow's foot notation is important.

**** Replace example screenshot below with your response**





Step 3

Create A Physical
Database

Step 3: Create A Physical Database

In this step, you will be turning your database model into a physical database.

You will:

- Create the database using SQL DDL commands
- Load the data into your database, utilizing flat file ETL
- Answer a series of questions using CRUD SQL commands to demonstrate your database was created and populated correctly

Submission

For this step, you will need to submit SQL files containing all DDL SQL scripts used to create the database.

You will also have to submit screenshots showing CRUD commands, along with results for each of the questions found in the starter template.

Hints

Your DDL script will be graded by running the code you submit. Please ensure your SQL code runs properly!

Foreign keys cannot be created on tables that do not exist yet, so it may be easier to create all tables in the database, then to go back and run modify statements on the tables to create foreign key constraints.

After running CRUD commands like update, insert, or delete, run a `SELECT*` command on the affected table, so the reviewer can see the results of the command.

DDL

Create a DDL SQL script capable of building the database you designed in Step 2

Hints

The DDL script will be graded by running the code you submit. Please ensure your SQL code runs properly.

Foreign keys cannot be created on tables that do not exist yet, so it may be easier to create all tables in the database, then to go back and run modify statements on the tables to create foreign key constraints.

Remember to submit the related SQL file as well, not just a screenshot (replace the below screenshot).

```
• use dbtest;

• CREATE TABLE TEAM (
    Team_ID INT,
    Team_NM varchar(100),
    FIELD_ID INT,
    primary key (Team_ID)
) ;
```

CRUD

- **Question 1: Return a list of employees with Job Titles and Department Names**

**** Replace example screenshot below with your response, and include the query in a SQL file**

```
select emp_name, j.job_nm, department_nm
from employee as e
join assignment as a
on e.emp_id = a.emp_id
join job as j
on a.job_id = j.job_id
join department as d
on a.department_id = d.department_id;
```

emp_name	job_nm	department_nm
Haifa Hajiri	Administrative Assistant	Distribution
Wendell Mobley	Administrative Assistant	Distribution
Michael Sperduti	Administrative Assistant	Distribution
Ashley Bergman	Administrative Assistant	Distribution
Carlos Lopez	Administrative Assistant	Distribution
Jason Wingard	Administrative Assistant	Distribution
Anita Deluise	Administrative Assistant	HQ
Joseph Donohue	Administrative Assistant	HQ
John Certa	Administrative Assistant	HQ
Raven Landis	Administrative Assistant	HQ
Randy Myers	Administrative Assistant	HQ
Danny Godiksen	Administrative Assistant	HQ
Raj Prudvi	Administrative Assistant	IT
Alexis Fitzpatrick	Administrative Assistant	IT
Ann Roberto	Administrative Assistant	IT
Danny Laxton	Administrative Assistant	IT
Laura McKenna	Administrative Assistant	IT
William Graf	Administrative Assistant	IT
Keith Ingram	Administrative Assistant	Product Development
Jill Fram	Administrative Assistant	Product Development
Alan Mecklet	Administrative Assistant	Product Development
Nicole Lee	Administrative Assistant	Product Development
Cheryl Pike	Administrative Assistant	Product Development
Krishna Burli	Administrative Assistant	Product Development
Kathy Abiskaroon	Administrative Assistant	Product Development
Aaron Richman	Administrative Assistant	Product Development
Muhammed Rubel	Administrative Assistant	Product Development
Tony Hughes	Administrative Assistant	Sales

CRUD

- **Question 2: Insert Web Programmer as a new job title**

**** Replace example screenshot below with your response, and include the query in a SQL file**

```
insert into job(job_nm)
values ('Web Programmer');
```

```
postgres=# insert into job(job_nm)
values ('Web Programmer');
INSERT 0 1
postgres=# select * from job;
 job_id |          job_nm
-----+-----
      1 | Shipping and Receiving
      2 | Sales Rep
      3 | Administrative Assistant
      4 | Design Engineer
      5 | Database Administrator
      6 | Software Engineer
      7 | Manager
      8 | Legal Counsel
      9 | President
     10 | Network Engineer
     11 | Web Programmer
(11 rows)
```

CRUD

- **Question 3: Correct the job title from web programmer to web developer**

**** Replace example screenshot below with your response, and include the query in a SQL file**

```
update job
set job_nm = 'Web Developer'
where job_nm = 'Web Programmer';
```

```
postgres=# update job
set job_nm = 'Web Developer'
where job_nm = 'Web Programmer';
UPDATE 1
postgres=# select * from job;
 job_id |          job_nm
-----+-----
      1 | Shipping and Receiving
      2 | Sales Rep
      3 | Administrative Assistant
      4 | Design Engineer
      5 | Database Administrator
      6 | Software Engineer
      7 | Manager
      8 | Legal Counsel
      9 | President
     10 | Network Engineer
     11 | Web Developer
(11 rows)
```

CRUD

- **Question 4: Delete the job title Web Developer from the database**

**** Replace example screenshot below with your response, and include the query in a SQL file**

```
delete from job
where job_nm = 'Web Developer';
```

```
postgres=# delete from job
where job_nm = 'Web Developer';
DELETE 1
postgres=# select * from job;
 job_id |          job_nm
-----+-----
      1 | Shipping and Receiving
      2 | Sales Rep
      3 | Administrative Assistant
      4 | Design Engineer
      5 | Database Administrator
      6 | Software Engineer
      7 | Manager
      8 | Legal Counsel
      9 | President
     10 | Network Engineer
(10 rows)
```

CRUD

- **Question 5: How many employees are in each department?**

```
select d.department_nm, count(e.emp_id) as num_emp
from employee as e
join assignment as a
on e.emp_id = a.emp_id
join department as d
on a.department_id = d.department_id
group by d.department_nm;
```

```
postgres=# select d.department_nm, count(e.emp_id) as num_emp
from employee as e
join assignment as a
on e.emp_id = a.emp_id
join department as d
on a.department_id = d.department_id
group by d.department_nm;
 department_nm | num_emp
-----+-----
Product Development |      70
HQ              |      13
Distribution     |      27
Sales           |      41
IT              |      54
(5 rows)
```

CRUD

- **Question 6: Write a query that returns current and past jobs (include employee name, job title, department, manager name, start and end date for position) for employee Toni Lembeck.**

```
select e.emp_name, j.job_nm, d.department_nm, m.manager_nm, start_dt, end_dt
from assignment as a
join employee as e
on a.emp_id = e.emp_id
join job as j
on a.job_id = j.job_id
join department as d
on a.department_id = d.department_id
join manager as m
on a.manager_id = m.manager_id
where emp_name = 'Toni Lembeck';
```

```
postgres=# select e.emp_name, j.job_nm, d.department_nm, m.manager_nm, start_dt, end_dt
from assignment as a
join employee as e
on a.emp_id = e.emp_id
join job as j
on a.job_id = j.job_id
join department as d
on a.department_id = d.department_id
join manager as m
on a.manager_id = m.manager_id
where emp_name = 'Toni Lembeck';
```

emp_name	job_nm	department_nm	manager_nm	start_dt	end_dt
Toni Lembeck	Database Administrator	IT	Jacob Lauber	2001-07-18	
Toni Lembeck	Network Engineer	IT	Jacob Lauber	1995-03-12	2001-07-17

(2 rows)

CRUD

- **Question 7: Describe how you would apply table security to restrict access to employee salaries using an SQL server.**

I would apply table security by creating a separate salaries table and only allowing certain domain logins (providing certain credentials and permissions) to be able to access the table. With postgresql I can grant and revoke privileges to specific users with specific roles and make sure they can only read from specific tables.



Step 4

Above and Beyond
(optional)

Step 4: Above and Beyond

This last step is called Above and Beyond. In this step, I have proposed 3 challenges for you to complete, which are above and beyond the scope of the project. This is a chance to flex your coding muscles and show everyone how good you really are.

These challenge steps will bring your project even more in line with a real-world project, as these are the kind of “finishing touches” that will make your database more usable. Imagine building a car without air conditioning or turn signals. Sure, it will work, but who would want to drive it.

I encourage you to take on these challenges in this course and any future courses you take. I designed these challenges to be a challenge to your current abilities, but I ensured they are not an unattainable challenge. Remember, these challenges are completely optional - you can pass the project by doing none of them, or just some of them, but I encourage you to at least attempt them!

Standout Suggestion 1

Create a view that returns all employee attributes; results should resemble initial Excel file

create view employees as

```
select e.emp_name, e.emp_email, j.job_nm, d.department_nm, l.location_nm,
       aa.address_nm, c.city_nm, ss.state_nm, m.manager_nm, ed.education_lvl,
       s.hire_dt, s.sal_val, start_dt, end_dt
```

from assignment as a

join employee as e

join education as ed

join job as j

join department as d

join manager as m

join location as l

join address as aa

join city as c

join state as ss

join salary as s

on a.emp_id = e.emp_id

on e.education_lvl_id = ed.education_lvl_id

on a.job_id = j.job_id

on a.department_id = d.department_id

on a.manager_id = m.manager_id

on a.location_id = l.location_id

on l.location_id = aa.address_id

on aa.city_id = c.city_id

on c.state_id = ss.state_id

on a.assignment_id = s.assignment_id;

local_postgres: 217 records on 'employees' table								
emp_name	emp_email	job_nm	department_nm	location_nm	address_nm	city_nm	state_nm	manager_nm
Haifa Hajiri	Haifa.Hajiri@TechCorp.com	Administrative Assistant	Distribution	West Coast	165 Broadway	New York City	NY	Allison Gentle
Wendell Mobley	Wendell.Mobley@TechCorp.com	Administrative Assistant	Distribution	West Coast	165 Broadway	New York City	NY	Allison Gentle
Michael Sperduti	Michael.Sperduti@TechCorp.com	Administrative Assistant	Distribution	West Coast	165 Broadway	New York City	NY	Allison Gentle
Ashley Bergman	Ashley.Bergman@TechCorp.com	Administrative Assistant	Distribution	Midwest	1300 Nicollet Mall	Minneapolis	MN	Allison Gentle
Carlos Lopez	Carlos.Lopez@TechCorp.com	Administrative Assistant	Distribution	East Coast	705 James Way	San Francisco	CA	Allison Gentle
Jason Wingard	Jason.Wingard@TechCorp.com	Administrative Assistant	Distribution	West Coast	165 Broadway	New York City	NY	Allison Gentle
Anita Deluise	Anita.Deluise@TechCorp.com	Administrative Assistant	HQ	HQ	422 Broadway	Nashville	TN	Tyrone Hutchison
Joseph Donohue	Joseph.Donohue@TechCorp.com	Administrative Assistant	HQ	HQ	422 Broadway	Nashville	TN	Tyrone Hutchison
John Certi	John.Certi@TechCorp.com	Administrative Assistant	HQ	HQ	422 Broadway	Nashville	TN	Tyrone Hutchison
Raven Landis	Raven.Landis@TechCorp.com	Administrative Assistant	HQ	HQ	422 Broadway	Nashville	TN	Tyrone Hutchison
Randy Myers	Randy.Myers@TechCorp.com	Administrative Assistant	HQ	HQ	422 Broadway	Nashville	TN	Tyrone Hutchison
Danny Godiksen	Danny.Godiksen@TechCorp.com	Administrative Assistant	HQ	HQ	422 Broadway	Nashville	TN	Tyrone Hutchison
Raj Prudvi	Raj.Prudvi@TechCorp.com	Administrative Assistant	IT	East Coast	705 James Way	San Francisco	CA	Jacob Lauber
Alexis Fitzpatrick	Alexis.Fitzpatrick@TechCorp.com	Administrative Assistant	IT	Midwest	1300 Nicollet Mall	Minneapolis	MN	Jacob Lauber
Ann Roberto	Ann.Roberto@TechCorp.com	Administrative Assistant	IT	Midwest	1300 Nicollet Mall	Minneapolis	MN	Jacob Lauber
Danny Laxton	Danny.Laxton@TechCorp.com	Administrative Assistant	IT	East Coast	705 James Way	San Francisco	CA	Jacob Lauber
Laura McKenna	Laura.McKenna@TechCorp.com	Administrative Assistant	IT	East Coast	705 James Way	San Francisco	CA	Jacob Lauber
William Graf	William.Graf@TechCorp.com	Administrative Assistant	IT	HQ	422 Broadway	Nashville	TN	Jacob Lauber
Keith Ingram	Keith.Ingram@TechCorp.com	Administrative Assistant	Product Development	HQ	422 Broadway	Nashville	TN	Conner Kinch
Jill Fram	Jill.Fram@TechCorp.com	Administrative Assistant	Product Development	HQ	422 Broadway	Nashville	TN	Conner Kinch
Alan Mecklet	Alan.Mecklet@TechCorp.com	Administrative Assistant	Product Development	East Coast	705 James Way	San Francisco	CA	Conner Kinch
Nicole Lee	Nicole.Lee@TechCorp.com	Administrative Assistant	Product Development	West Coast	165 Broadway	New York City	NY	Conner Kinch
Cheryl Pike	Cheryl.Pike@TechCorp.com	Administrative Assistant	Product Development	South	1 Tech ABC Corp Way	Dallas	TX	Conner Kinch

local_postgres: 217 records on 'employees' table									
nm	address_nm	city_nm	state_nm	manager_nm	education_lvl	hire_dt	sal_val	start_dt	end_dt
st	165 Broadway	New York City	NY	Allison Gentle	No College	2003-12-17	\$47,418.00	2003-12-17	NULL
st	165 Broadway	New York City	NY	Allison Gentle	Some College	2013-11-27	\$28,969.00	2013-11-27	NULL
st	165 Broadway	New York City	NY	Allison Gentle	Associates Degree	2014-06-20	\$43,778.00	2014-06-20	NULL
st	1300 Nicollet Mall	Minneapolis	MN	Allison Gentle	No College	2009-03-01	\$41,090.00	2009-03-01	NULL
st	705 James Way	San Francisco	CA	Allison Gentle	No College	2014-05-23	\$35,825.00	2014-05-23	NULL
st	165 Broadway	New York City	NY	Allison Gentle	Some College	2006-01-11	\$34,809.00	2006-01-11	NULL
st	422 Broadway	Nashville	TN	Tyrone Hutchison	Associates Degree	1995-06-01	\$45,325.00	1995-06-01	NULL
st	422 Broadway	Nashville	TN	Tyrone Hutchison	No College	2007-12-27	\$40,647.00	2007-12-27	NULL
st	422 Broadway	Nashville	TN	Tyrone Hutchison	Some College	2007-08-29	\$51,633.00	2007-08-29	NULL
st	422 Broadway	Nashville	TN	Tyrone Hutchison	Associates Degree	2016-04-21	\$42,313.00	2016-04-21	NULL
st	422 Broadway	Nashville	TN	Tyrone Hutchison	Associates Degree	2020-01-24	\$34,084.00	2020-01-24	NULL
st	422 Broadway	Nashville	TN	Tyrone Hutchison	Associates Degree	2010-12-15	\$28,373.00	2010-12-15	NULL
st	705 James Way	San Francisco	CA	Jacob Lauber	Associates Degree	2017-10-03	\$51,052.00	2017-10-03	NULL
st	1300 Nicollet Mall	Minneapolis	MN	Jacob Lauber	No College	2013-02-19	\$39,450.00	2013-02-19	NULL
st	1300 Nicollet Mall	Minneapolis	MN	Jacob Lauber	No College	2009-10-01	\$37,136.00	2009-10-01	NULL
st	705 James Way	San Francisco	CA	Jacob Lauber	Some College	1999-10-24	\$34,677.00	1999-10-24	NULL
st	705 James Way	San Francisco	CA	Jacob Lauber	Associates Degree	1997-10-15	\$45,210.00	1997-10-15	NULL
st	422 Broadway	Nashville	TN	Jacob Lauber	Bachelors Degree	2005-05-26	\$50,043.00	2005-05-26	NULL
st	422 Broadway	Nashville	TN	Conner Kinch	Some College	1996-04-14	\$48,910.00	1996-04-14	NULL
st	422 Broadway	Nashville	TN	Conner Kinch	Associates Degree	2007-12-16	\$50,462.00	2007-12-16	NULL

Standout Suggestion 2

Create a stored procedure with parameters that returns current and past jobs (include employee name, job title, department, manager name, start and end date for position) when given an employee name.

```
CREATE OR REPLACE FUNCTION job_history(p_emp_name VARCHAR)
RETURNS TABLE (
  emp_name VARCHAR,
  job_nm VARCHAR,
  department_nm VARCHAR,
  manager_nm VARCHAR,
  start_dt DATE,
  end_dt DATE
)
LANGUAGE SQL
AS $$
SELECT
  e.emp_name,
  j.job_nm,
  d.department_nm,
  m.manager_nm,
  a.start_dt,
  a.end_dt
FROM assignment AS a
JOIN employee AS e
ON a.emp_id = e.emp_id
JOIN job AS j
ON a.job_id = j.job_id
JOIN department AS d
ON a.department_id = d.department_id
JOIN manager AS m
ON a.manager_id = m.manager_id
WHERE e.emp_name = p_emp_name;
$$;

SELECT *
FROM job_history('Toni Lembeck');
```

emp_name	job_nm	department_nm	manager_nm	start_dt	end_dt
Toni Lembeck	Database Administrator	IT	Jacob Lauber	2001-07-18	
Toni Lembeck	Network Engineer	IT	Jacob Lauber	1995-03-12	2001-07-17
(2 rows)					

Standout Suggestion 3

Implement user security on the restricted salary attribute.

Create a non-management user named **NoMgr**. Show the code of how you would grant access to the database, but revoke access to the salary data

```
grant select
on all tables in hr-db public
to nomgr;
#login as nomgr and check access
#go back to admin role
revoke select
on salary
from nomgr;
#login as nomgr
select * from salary limit 10;
```

```
postgres=# grant select
on all tables in schema public
to nomgr;
GRANT
postgres=# exit
postgres@r-db:~$ psql -h localhost -U nomgr postgres
psql (17.4 (Debian 17.4-1.pgdg120+2))
Type "help" for help.

postgres=> select current_user;
 current_user
-----
 nomgr
(1 row)

postgres=> select * from salary limit 10;
 sal_id | assignment_id | sal_val | hire_dt
-----+-----+-----+-----
    1   |              1 | $47,418.00 | 2003-12-17
    2   |              2 | $28,969.00 | 2013-11-27
    3   |              3 | $43,778.00 | 2014-06-20
    4   |              4 | $41,090.00 | 2009-03-01
    5   |              5 | $35,825.00 | 2014-05-23
    6   |              6 | $34,809.00 | 2006-01-11
    7   |              7 | $45,325.00 | 1995-06-01
    8   |              8 | $40,647.00 | 2007-12-27
    9   |              9 | $51,633.00 | 2007-08-29
   10   |             10 | $42,313.00 | 2016-04-21
(10 rows)

postgres=> exit
postgres@r-db:~$ psql
psql (17.4 (Debian 17.4-1.pgdg120+2))
Type "help" for help.
```

```
postgres=# revoke select
on salary
from nomgr;
REVOKE
postgres=# select current_user;
 current_user
-----
 postgres
(1 row)

postgres=# exit
postgres@r-db:~$ psql -h localhost -U nomgr postgres
psql (17.4 (Debian 17.4-1.pgdg120+2))
Type "help" for help.

postgres=> select current_user;
 current_user
-----
 nomgr
(1 row)

postgres=> select * from salary limit 10;
ERROR: permission denied for table salary
postgres=>
```